

Atividade 3.1

Parte 1

Multiplicação de matrizes

Redes neurais

Siga o mesmo procedimento de entrega das atividades anteriores.

Coloque em Jupyter's diferentes a Parte 1 e a Parte 2 no mesmo repositório.

A1

Implemente uma rede neural totalmente conectada com **duas camadas ocultas e uma camada de saída**, utilizando PyTorch. Cada camada oculta deve ser representada por uma **matriz 4×4** , e a camada de saída por uma **matriz 4×1** . Utilize a função de ativação **softplus** entre as camadas.

A rede deve ser treinada com o **conjunto de dados Iris**, utilizando apenas os atributos **sepal** e **petal length** como entradas, e apenas duas classes: **Setosa** e **Versicolor**.

- Normalize os dados de entrada.
- Use partição simples da base.
- Inicialize três matrizes de pesos conforme explicado em sala.
- Realize a propagação direta (forward pass) usando apenas `torch.matmul` e [`torch.nn.functional.softplus`](#)
- Utilize o erro quadrático médio com função de perda e SGD como otimizador.
- Compare os resultados com [`ch02.ipynb`](#)

B1

Neste exercício, você continuará utilizando a mesma arquitetura de rede neural e configuração do conjunto de dados do Problema 1:

- Duas características de entrada: comprimento da sépala e comprimento da pétala;
- Duas classes de saída: setosa e versicolor;

- Duas camadas ocultas. Cada camada oculta deve ser representada por uma **matriz 4×4**, e a camada de saída por uma **matriz 4×1**. Utilize a função de ativação **softplus** entre as camadas;
- Função de perda: MSELoss.

Seu objetivo é treinar a rede utilizando mini-batches, em vez de alimentar a rede com todo o conjunto de dados de uma vez (full batch):

- Divida o conjunto de dados em 4 lotes (batches).
- Treine a rede com as operações necessárias incluindo `backward()` e `step()` para atualizar os pesos.

C1

Entender o código até esta figura:

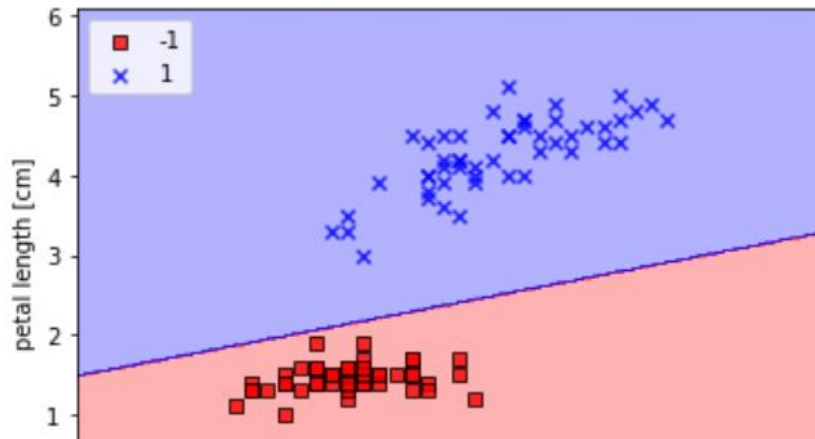
[ch02.ipynb](#)

Será pedido para ser explicado em sala.

```
plot_decision_regions(X, y, classifier=ppn)
plt.xlabel('sepal length [cm]')
plt.ylabel('petal length [cm]')
plt.legend(loc='upper left')
```

```
# plt.savefig('images/02_08.png', dpi=300)
plt.show()
```

<ipython-input-13-0999be6beb6e>:24: UserWarning: You
plt.scatter(x=X[y == c1, 0],



D1

Adapte o código anterior para ser executado para os dados sintéticos abaixo:

Linearly separable.ipynb

+ Código

+ Texto



```
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt

X, y = make_classification(n_samples=100, n_features=2, n_classes=2,
                          n_clusters_per_class=1, n_redundant=0, random_state=42)

plt.scatter(X[:, 0], X[:, 1], c=y, cmap='bwr')
plt.show()
```

E1

Gradient Descent, Step-by-Step

Transforme em código Python o procedimento do slide 301 ao slide 325. O código deve rodar até o final, ou seja, até a condição de parada.

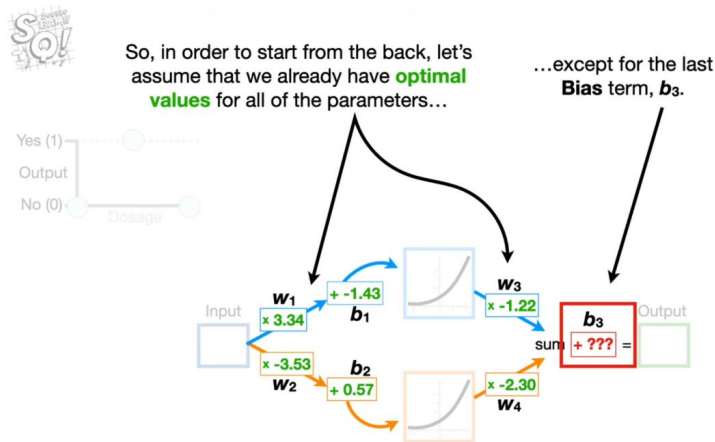
Após finalizado o código acima e extraídos seus resultados, adapte o código para ele rodar como o Gradiente Descendente Estocástico. Compare os resultados.

F1

Neural Networks by StatQuest - Backpropagation Main Ideas

Duas formas de resolver essa atividade. Escolha uma delas.

A primeira é usar o PyTorch e resolver a rede do slide 77.

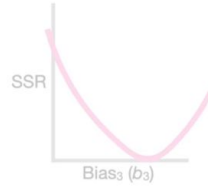


A segunda é converter para código em uma apresentação no Jupyter o slide 235 até o slide 280. Ser bastante didático. Não precisa ser um linha de código por slide.



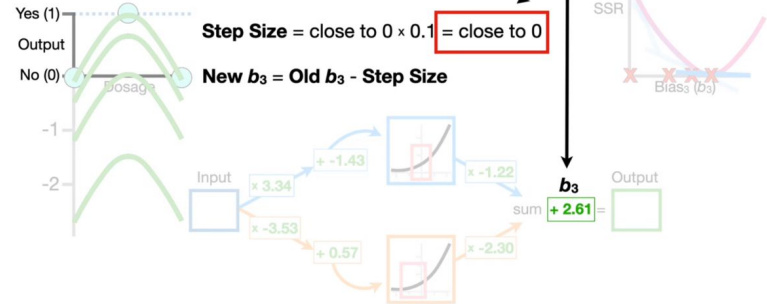
$$\frac{d SSR}{d b_3} = -2 \times (\text{Observed}_1 - \text{Predicted}_1) \times 1 \\ + -2 \times (\text{Observed}_2 - \text{Predicted}_2) \times 1 \\ + -2 \times (\text{Observed}_3 - \text{Predicted}_3) \times 1$$

Anyway, first, we expand the summation.



$$\frac{d SSR}{d b_3} = -2 \times (0 - 0.01) \\ + -2 \times (1 - 1.01) \\ + -2 \times (0 - 0) = \text{close to } 0$$

...we decide that **2.61** is the optimal value for **b₃**.

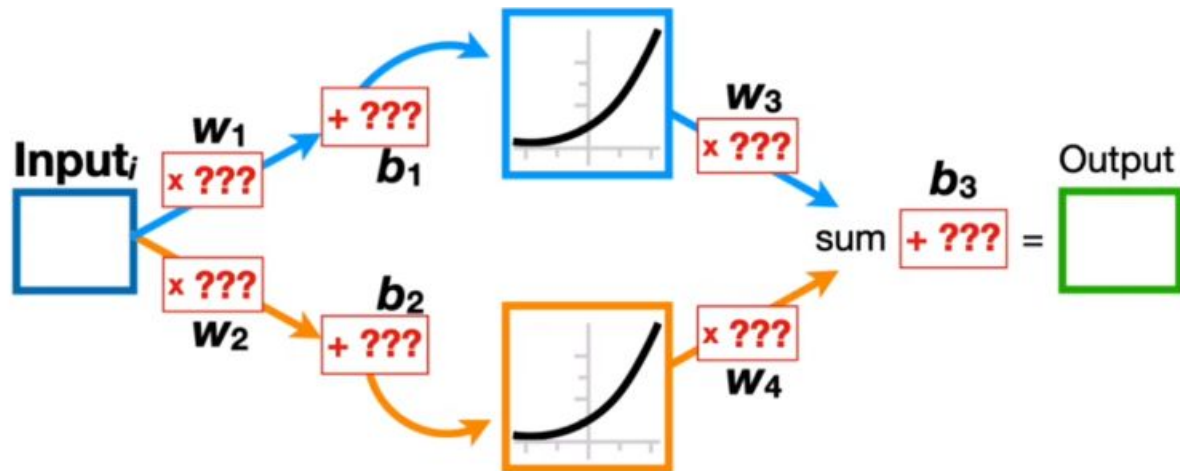


G1

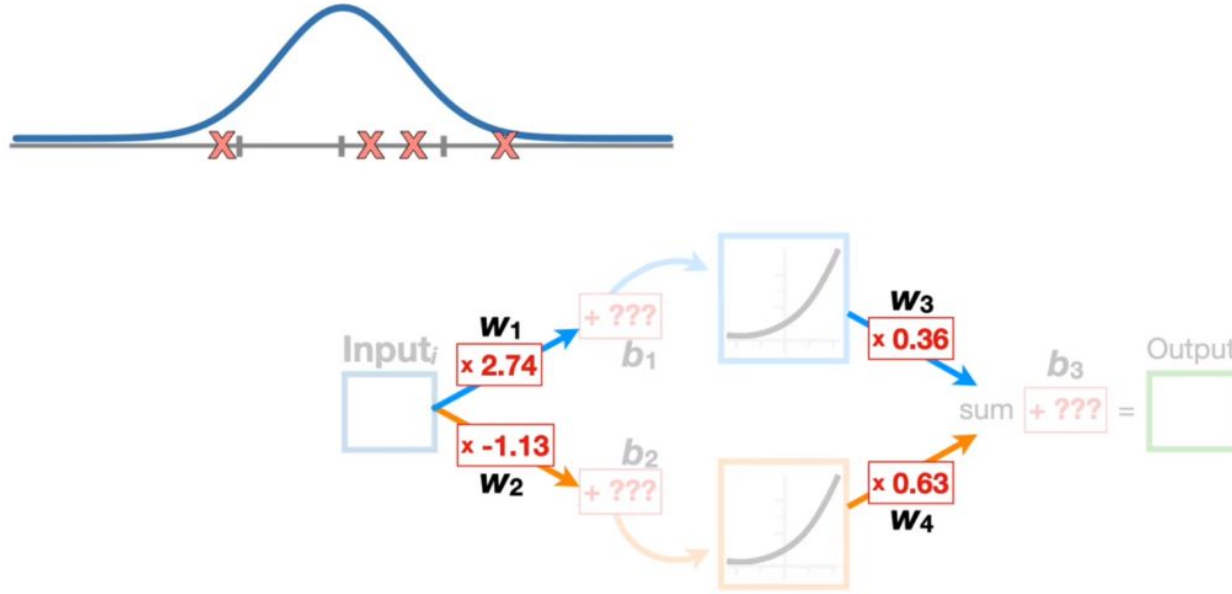
[Backpropagation Details Pt. 2: Going bonkers with The Chain Rule](#)

Resolva a rede do slide 176 com código em Python que deve seguir o procedimento a partir deste slide para o backpropagation. Seja bastante didático.

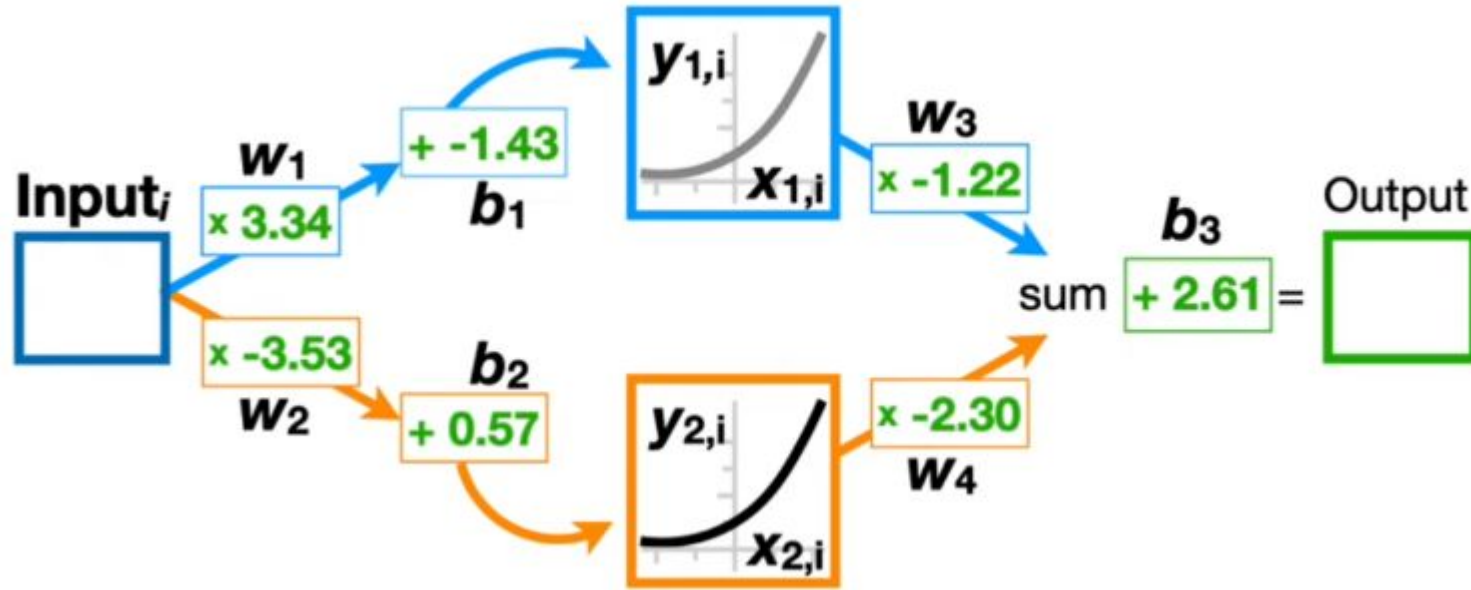
Resolva **também** a mesma rede com [PyTorch](#).



Mostre o procedimento de inicialização de forma explícita e funcional para o primeiro caso.



Simule o procedimento até ele convergir.



Parte 2

Explorando redes neurais

CNN

RNN

Exploring word embeddings

[Neural Networks Pt. 3: ReLU In Action.pptx](#)

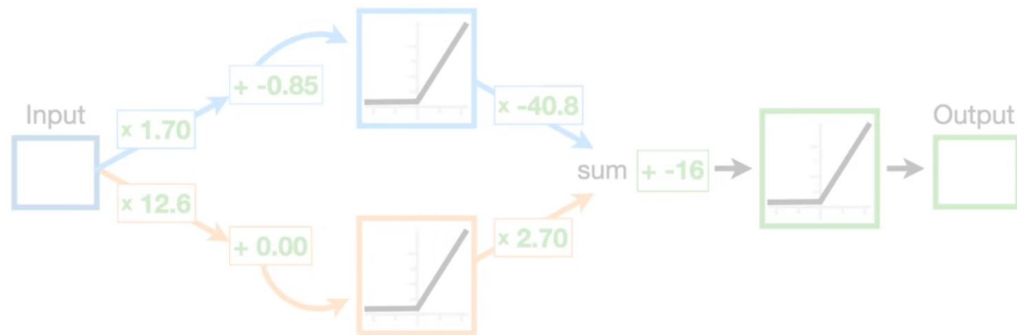
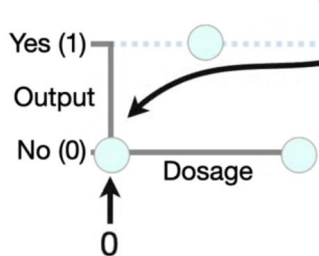
[Neural Networks Pt. 4: Multiple Inputs and Outputs](#)

A2

Treine essa rede no PyTorch considerando os dados mostrados nos slides. Mostre a saída para três valores depois do treinamento. Compare a rede obtida simulando a rede do slide no PyTorch.



Remember: To keep the math simple, let's assume **Dosages** go from **0** (low) to **1** (high).

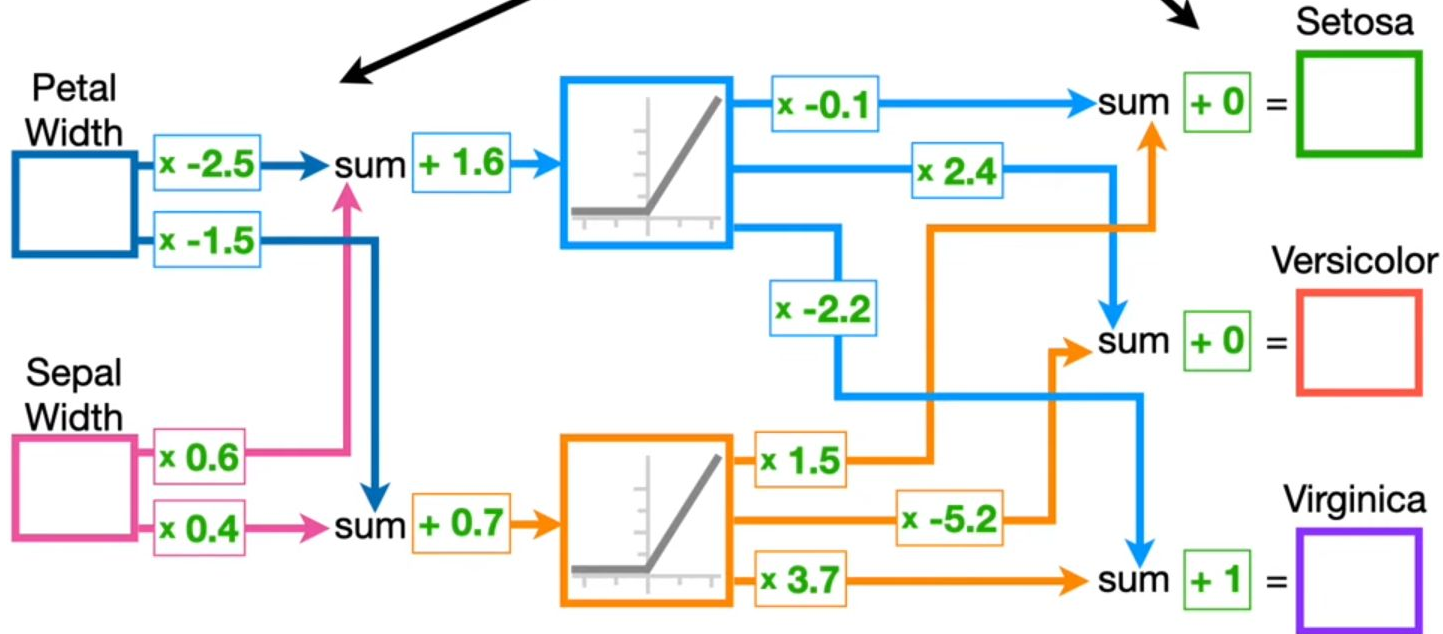


B2

Simule a rede mostrada no próximo slide no PyTorch. Obtenha a saída para cinco valores de cada feature. Retire do dataset da Iris. Em seguida aplique as funções `argmax` e `softmax`, e interprete os resultados. Calcule a cross entropy.



Now, this **Neural Network** may look really fancy...



C2

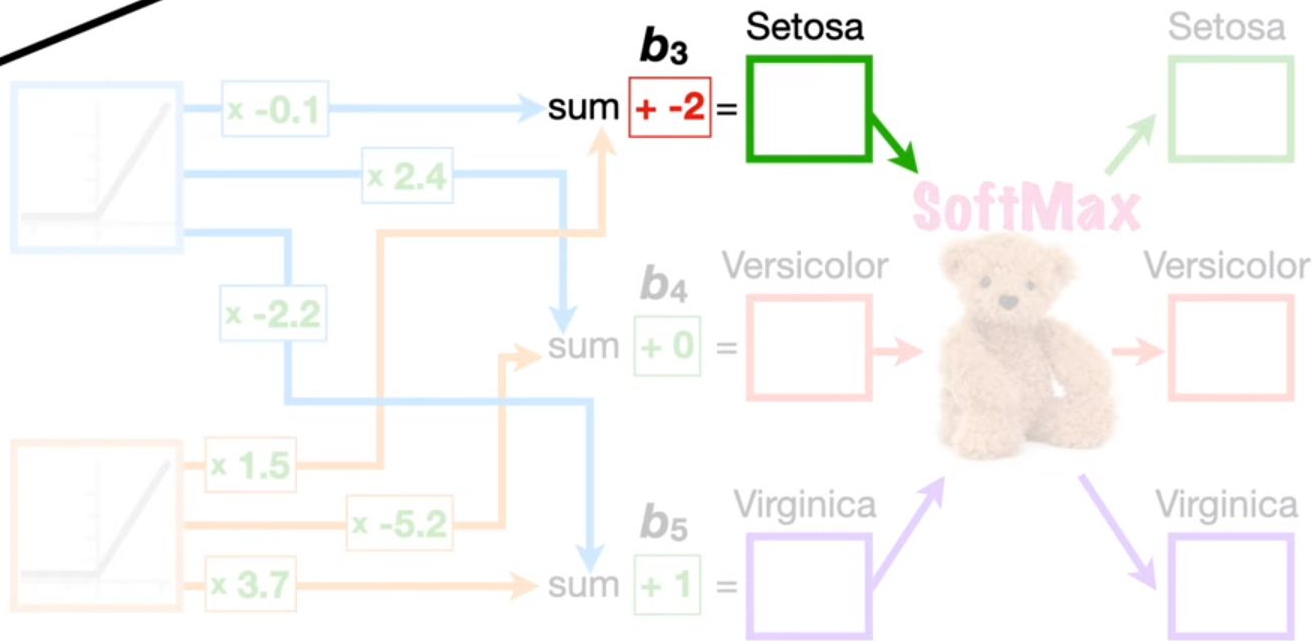
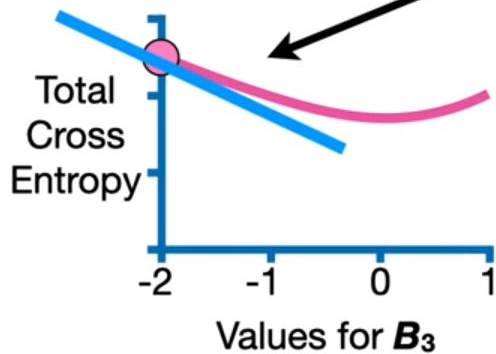
Reproduza com código os slides 311 a 375. Não use PyTorch. Diferente da apresentação, continue gerando steps do gradiente e continue o processo de otimização de b3 (slide 369) até que ele convirja.

[Neural Networks Part 7: Cross Entropy
Derivatives and Backpropagation.pptx](#)



$$\frac{d \text{ Cross Entropy}}{d b_3} = \sum_{i=1}^{n=3} \frac{d}{d b_3} \text{CE}(\text{Predicted}_i)$$

And that starts with the derivative of the **Cross Entropy** with respect to b_3 .



D2

Chapter 2 - Gradient descent, how neural networks learn

Conforme o procedimento do slide 23 em diante, treine no PyTorch a rede dada no slide seguinte, com três padrões em uma figura com 3x3 pixels. Vocês devem definir os padrões a serem aprendidos.

[Simple NN for Activity.ipynb](#)

```
import torch
import torch.nn as nn
import torch.optim as optim

class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(9, 4) # First hidden layer
        self.fc2 = nn.Linear(4, 4) # Second hidden layer
        self.fc3 = nn.Linear(4, 3) # Output layer
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x) # No activation function at the output layer (depends on task)
        return x

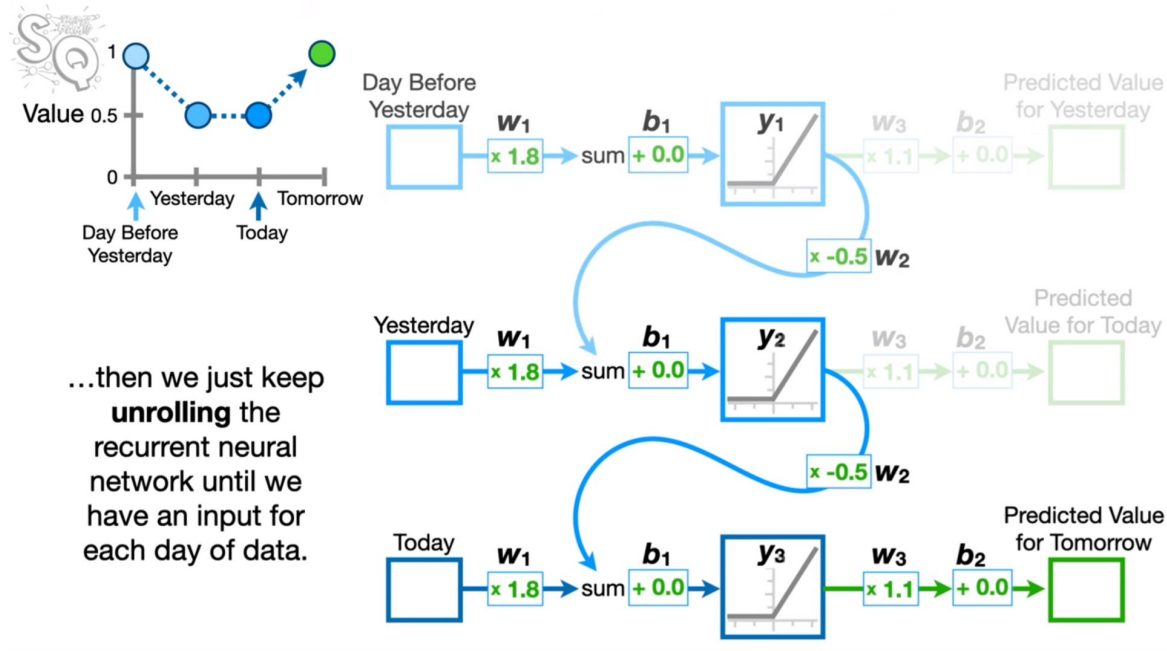
# Example usage
model = SimpleNN()
print(model)
```

Recurrent Neural Networks and word embeddings

Transformers

E2

Working with the unrolled neural network in slide 189, create code that simulates this process



Explain the main issues with recurrent neural networks in at least three slides.

F2

Exploring word embeddings: Training Neural Networks with Different Embedding Dimensions

Construct two phrases as training data, each containing three words (tokens). Train two neural networks in PyTorch: the first with an embedding dimension of 2 and the second with an embedding dimension of 4.

Explain each model step-by-step and compare how the choice of embedding size affects the learned representations of words. For the 2D case, plot the embeddings before and after training.

Model 1: Uses an embedding dimension of 2 (meaning each word will be mapped to a 2D vector).

Model 2: Uses an embedding dimension of 4 (each word will be mapped to a 4D vector).